

i2i Academy

Training Document

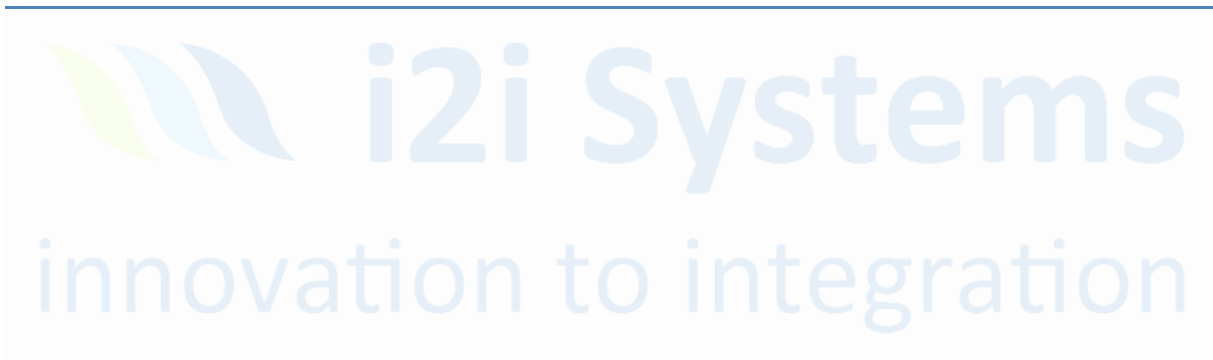
Topic	Applied Image Processing
Document Name	AppliedImageProcessing

Copyright of  i2i Systems Turkey 2026

The copyright in this work is vested in i2i Systems Turkey and the information contained herein is confidential. This work (either in whole or in part) must not be modified, reproduced, disclosed or disseminated to others or used for purposes other than that for which it is supplied, without the prior written permission of i2i Systems Turkey. If this work or any part hereof is furnished to a third party by virtue of a contract with that party, use of this work by such party shall be governed by the express contractual terms between the i2i Systems Turkey which is a party to that contract and the said party.

Document History

Date	Author	Ver	Comments
21.04.2026	i2i Academy	1.0	Initial Creation.



Document History.....	2
1. Difficulty Level.....	4
2. General Requirements	5
2.1. Code Quality.....	5
2.2. Documentation and Homework Submission	5
2.3. Code Management and Portfolio Building (GitHub).....	5
2.4. Submission Process (Google Classroom)	5
2.5. Tracking and Reporting (Google Sheets)	5
2.6. Academic Integrity	6
3. Description.....	7
3.1. Objective	7
3.1.1. Theoretical Knowledge.....	7
3.1.2. Practical Application.....	7
4. Solution.....	8

1. Difficulty Level

Beginner	Junior	Mid	Senior	Expert
•	•	•	•	•





2. General Requirements

Compliance with the following standards is mandatory for all homework assignments. These rules aim to ensure a fair evaluation process and instill professional working discipline.

2.1. Code Quality

Code must be readable, modular, and compliant with Clean Code principles. Avoid redundant comments and use meaningful naming conventions for variables and functions.

2.2. Documentation and Homework Submission

Every assignment must be documented with clear explanations and screenshots. Upon completion of the tasks, the designated Solution section (Section 4) must be filled out completely. The final step is to upload and submit this completed document exclusively via Google Classroom.

2.3. Code Management and Portfolio Building (GitHub)

Every homework assignment must have a "Public" repository on GitHub. Repositories must be named following the i2i-Academy-[HomeworkName]-[HomeworkNo] (e.g., i2i-Academy-ConnectingACloudService-1) format. Code must be uploaded at regular intervals with meaningful commit messages; avoid "bulk uploads" at the end of the project.

- **Important Note:** The primary goal of using GitHub is to help you build a professional portfolio for your future career. Therefore, **do not upload this homework document** (instructions, screenshots, etc.) to your GitHub repository. You may explain the purpose and scope of the project in your README.md file, but the repository must contain only your actual source code and configuration files.

2.4. Submission Process (Google Classroom)

Official notification of homework completion must be made exclusively via Google Classroom. The GitHub repository link must be provided in the assignment submission, and the process must be finalized by clicking the "Turn In" button. Submissions made after the specified deadline may not be evaluated unless prior approval is obtained.

2.5. Tracking and Reporting (Google Sheets)

For each completed task, the checkbox of the corresponding homework must be checked in the shared Google Sheet. Ensuring consistency between the Classroom submission and the Google Sheet update is the student's responsibility.

2.6. Academic Integrity

All submitted work must be original. Copying code from other students or uncredited sources is strictly prohibited.



3. Description

3.1. Objective

While Deep Learning gets all the hype, true engineers know the power of classical image processing. Before an AI can read text, the image must be filtered and prepared. Your task is to act as an AI Engineer building an Automated License Plate Recognition (ALPR) module. You will clean the image, locate the rectangular plate mathematically, and extract the characters.

3.1.1. Theoretical Knowledge

Briefly explain the following questions **using a maximum of 5 sentences total**:

- What is the main difference between "Computer Vision" and classical "Image Processing"?
- Why is it almost always necessary to convert an image to Grayscale (black and white) before applying edge detection algorithms?
- What does an OCR (Optical Character Recognition) engine do?

3.1.2. Practical Application

1. Find or take a clear, well-lit image of a car where the license plate is visible.
2. Load the image using OpenCV. Convert it to grayscale, apply a blur to reduce noise, and use an edge detection method (e.g., Canny Edge Detection).
3. Find the contours (shapes) in your processed image. Write logic to isolate the contour that looks like a license plate (typically a rectangular shape). Crop this specific section from the original image.
4. Pass the cropped image of the license plate to an OCR library (like EasyOCR or Pytesseract) to extract the text.
5. Print the final recognized license plate string to your terminal.

Please upload your clean, well-commented Python code and a short screen recording demonstrating that it works.

4. Solution

- 1. Computer Vision vs. Image Processing:** Image processing modifies or enhances an image mathematically (like applying filters), whereas computer vision analyzes that image to understand its content and make decisions like a human brain.
- 2. Why Grayscale for Edge Detection:** Converting an image to grayscale removes unnecessary color channels, reducing complexity so the algorithm can quickly find edges based purely on sudden changes in light intensity.
- 3. What an OCR Engine Does:** An OCR engine scans an image containing text and translates those visual shapes into digital, machine-readable text data that a computer can process.

```
PS C:\Users\ahmet> python -u "c:\Users\ahmet\Desktop\OCR_i2i\OCR.py"
İşlem başlatıldı, resim işleniyor...
Using CPU. Note: This module is much faster with a GPU.

[BAŞARILI] Tespit Edilen Plaka: 34AVEC01TRA
PS C:\Users\ahmet> █
```

2. Practical Application Steps & Code Explanations

Step 1: Image Preprocessing (Görüntü Ön İşleme) To prepare the image for contour detection, I first loaded the original image using OpenCV. Then, I converted it to grayscale (cv2.cvtColor) to reduce complexity. To remove noise while keeping the edges sharp, I applied a Bilateral Filter (cv2.bilateralFilter). Finally, I used the Canny Edge Detection algorithm (cv2.Canny) to highlight the structural boundaries in the image.



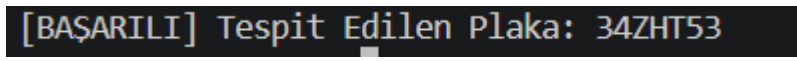
- Screenshot 1:

Step 2: Contour Detection and Cropping Using the edge-detected image, I utilized cv2.findContours to find all shapes. I sorted these contours by area (largest first) and used cv2.approxPolyDP to check their geometric approximations. The logic is simple: a license plate is a rectangular object, so the algorithm isolates the first large contour with exactly 4 corners. Once found, I calculated its bounding box (cv2.boundingRect) and cropped this specific region from the original image.



Screenshot 2:

Step 3: OCR Process Before passing the cropped plate to the OCR engine, I applied resizing and Otsu Thresholding (cv2.THRESH_OTSU) to convert the cropped plate into a pure black-and-white binary image. This step significantly increased the accuracy of the OCR. I then passed this binary image to EasyOCR, restricting the allowlist to only uppercase letters and numbers to prevent punctuation errors.

- Screenshot 3: 

3. Project Links

- **GitHub Repository Link:** <https://github.com/ahmetikolik/i2i-Academy-AppliedImageProcessing-6.git>

